A Style-Based Generator Architecture for Generative Adversarial Networks

Karras, Laine, and Aila, *CVPR*, 2019

Date: 2021.03.21

Members: 김상현, 김선욱, 신정호

Presenter: 허다운

Contents

- Contribution
- Background
- Style-based Generator
- Quality of generated images
- Properties of the style-based generator
- Disentanglement studies & Quantifying methods
- Conclusion

Contribution

- Motivated by style transfer literature, researcher re-design the generator architecture in a way that exposes novel ways to control the image synthesis process.

- Proposed new generator
  - The new architecture leads to an automatically learned, unsupervised seperation of high-level attributes
  - Stochastic variation in the generated images (e.g., freckles, hair)
  - Intuitive, scale-specific control of the synthesis
    (i.e. directly controlling the strength of image features at different scales)

- Proposed two new methods to quantify interpolation quality and disentanglement

- Introduced a new, highly varied and high-quality dataset of human faces

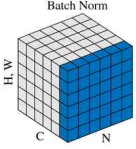
Background: Several Normalization Methods

- Batch normalization [2]

$$\mathrm{BN}(x) = \gamma\left(\frac{x-\mu(x)}{\sigma(x)}\right) + \beta \quad (1)$$

$$\mu_c(x) = \frac{1}{NHW}\sum_{n=1}^{N}\sum_{h=1}^{H}\sum_{w=1}^{W} x_{nchw} \quad (2)$$

$$\sigma_c(x) = \sqrt{\frac{1}{NHW}\sum_{n=1}^{N}\sum_{h=1}^{H}\sum_{w=1}^{W}(x_{nchw}-\mu_c(x))^2 + \epsilon} \quad (3)$$



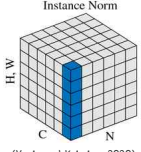
(Yuxin and Kaiming, 2020)

Instance normalization [3]

$$\mathrm{IN}(x) = \gamma\left(\frac{x-\mu(x)}{\sigma(x)}\right) + \beta \quad (4)$$

$$\mu_{nc}(x) = \frac{1}{HW}\sum_{h=1}^{H}\sum_{w=1}^{W} x_{nchw} \quad (5)$$

$$\sigma_{nc}(x) = \sqrt{\frac{1}{HW}\sum_{h=1}^{H}\sum_{w=1}^{W}(x_{nchw}-\mu_{nc}(x))^2 + \epsilon} \quad (6)$$



(Yuxin and Kaiming, 2020)

- Conditional Instance Normalization

$$\mathrm{CIN}(x;s) = \gamma^s\left(\frac{x-\mu(x)}{\sigma(x)}\right) + \beta^s \quad (7)$$

- Adaptive Instance Normalization

$$\mathrm{AdaIN}(x,y) = \sigma(y)\left(\frac{x-\mu(x)}{\sigma(x)}\right) + \mu(y) \quad (8)$$

Background: What is Progressive Growing GAN? [5]

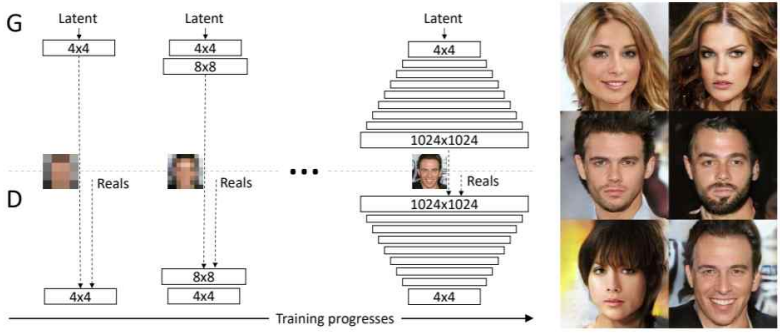
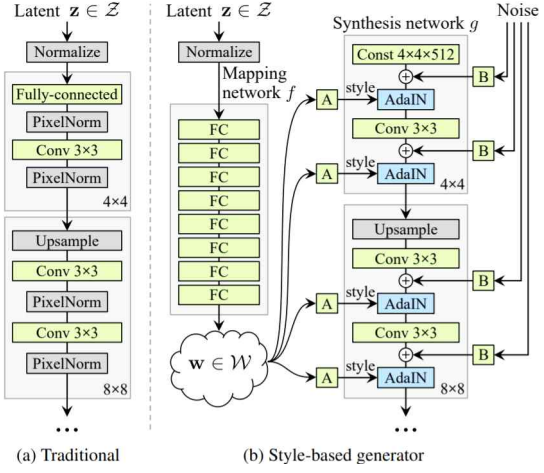
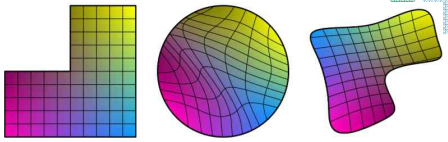


Figure 1: Our training starts with both the generator (G) and discriminator (D) having a low spatial resolution of 4×4 pixels. As the training advances, we incrementally add layers to G and D, thus increasing the spatial resolution of the generated images. All existing layers remain trainable throughout the process. Here $\boxed{N \times N}$ refers to convolutional layers operating on $N \times N$ spatial resolution. This allows stable synthesis in high resolutions and also speeds up training considerably. One the right we show six example images generated using progressive growing at 1024×1024.

Style-based Generator





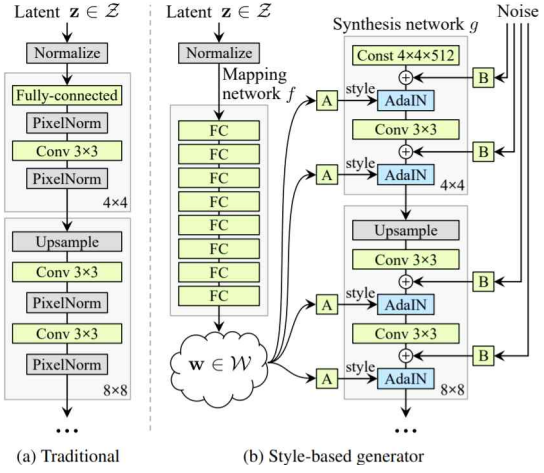
(a) Distribution of features in training set

(b) Mapping from $\mathcal{Z}$ to features

(c) Mapping from $\mathcal{W}$ to features

Figure 1. While a traditional generator [5] feeds the latent code though the input layer only, we first map the input to an intermediate latent space W, which then controls the generator through adaptive instance normalization (AdaIN) at each convolution layer. Gaussian noise is added after each convolution, before evaluating the nonlinearity. Here "A" stands for a learned affine transform, and "B" applies learned per-channel scaling factors to the noise input. The mapping network f consists of 8 layers and the synthesis network g consists of 18 layers—two for each resolution (4^2 – 1024^2). The output of the last layer is converted to RGB using a separate 1 × 1 convolution, similar to Karras et al. [5]. Our generator has a total of 26.2M trainable parameters, compared to 23.1M in the traditional generator.

Style-based Generator



(a) Traditional (b) Style-based generator

$$\text{AdaIN}(x, y) = \sigma(y)\left(\frac{x - \mu(x)}{\sigma(x)}\right) + \mu(y) \qquad (8)$$

Figure 1. While a traditional generator [5] feeds the latent code though the input layer only, we first map the input to an intermediate latent space W, which then controls the generator through adaptive instance normalization (AdaIN) at each convolution layer. Gaussian noise is added after each convolution, before evaluating the nonlinearity. Here "A" stands for a learned affine transform, and "B" applies learned per-channel scaling factors to the noise input. The mapping network f consists of 8 layers and the synthesis network g consists of 18 layers— two for each resolution (4^2 – 1024^2). The output of the last layer is converted to RGB using a separate 1 × 1 convolution, similar to Karras et al. [5]. Our generator has a total of 26.2M trainable parameters, compared to 23.1M in the traditional generator.

Questions?

Quality of generated images

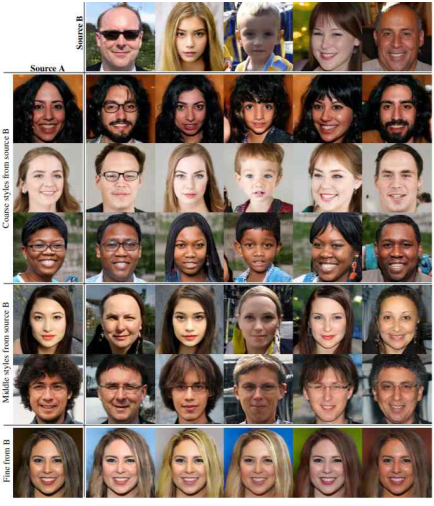
Method	CelebA-HQ	FFHQ
A Baseline Progressive GAN [30]	7.79	8.04
B + Tuning (incl. bilinear up/down)	6.11	5.25
C + Add mapping and styles	5.34	4.85
D + Remove traditional input	5.07	4.88
E + Add noise inputs	**5.06**	4.42
F + Mixing regularization	5.17	**4.40**

Table 1. Fréchet inception distance (FID) for various generator designs (lower is better). In this paper we calculate the FIDs using 50,000 images drawn randomly from the training set, and report the lowest distance encountered over the course of training.

Figure 2. Uncurated set of images produced by our style-based generator (config F) with the FFHQ dataset. Here we used a variation of the truncation trick with $\psi = 0.7$ for resolutions $4^2 - 32^2$.

Figure 7. The FFHQ dataset offers a lot of variety in terms of age, ethnicity, viewpoint, lighting, and image background.

Properties of the style-based generator: Style mixing



Mixing regularization	Number of latents during testing			
	1	2	3	4
E 0%	4.42	8.22	12.88	17.41
50%	4.41	6.10	8.71	11.61
F 90%	**4.40**	**5.11**	6.88	9.03
100%	4.83	5.17	**6.63**	**8.40**

Table 2. FIDs in FFHQ for networks trained by enabling the mixing regularization for different percentage of training examples. Here we stress test the trained networks by randomizing 1...4 latents and the crossover points between them. Mixing regularization improves the tolerance to these adverse operations significantly. Labels E and F refer to the configurations in Table 1.

Figure 3. Two sets of images were generated from their respective latent codes (sources A and B); the rest of the images were generated by copying a specified subset of styles from source B and taking the rest from source A. Copying the styles corresponding to coarse spatial resolutions (4^2– 8^2) brings high-level aspects such as pose, general hair style, face shape, and eyeglasses from source B, while all colors (eyes, hair, lighting) and finer facial features resemble A. If we instead copy the styles of middle resolutions (16^2–32^2) from B, we inherit smaller scale facial features, hair style, eyes open/closed from B, while the pose, general face shape, and eyeglasses from A are preserved. Finally, copying the fine styles (64^2– 1024^2) from B brings mainly the color scheme and microstructure.

Properties of the style-based generator: Stochastic variation

(a) Generated image (b) Stochastic variation (c) Standard deviation

Figure 4. Examples of stochastic variation. (a) Two generated images. (b) Zoom-in with different realizations of input noise. While the overall appearance is almost identical, individual hairs are placed very differently. (c) Standard deviation of each pixel over 100 different realizations, highlighting which parts of the images are affected by the noise. The main areas are the hair, silhouettes, and parts of background, but there is also interesting stochastic variation in the eye reflections. Global aspects such as identity and pose are unaffected by stochastic variation.

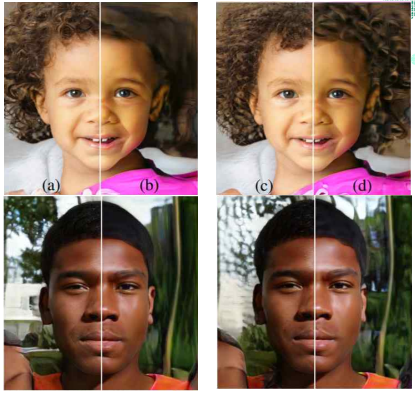
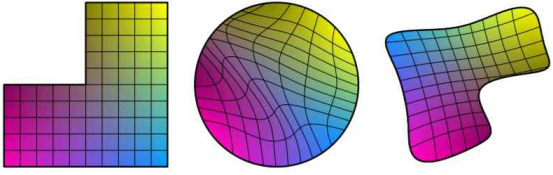


Figure 5. Effect of noise inputs at different layers of our generator. (a) Noise is applied to all layers. (b) No noise. (c) Noise in fine layers only (64^2– 1024^2): brings out the finer curls of hair, finer background detail, and skin pores. (d) Noise in coarse layers only (4^2– 32^2): causes large-scale curling of hair and appearance of larger background features

Disentanglement studies & Quantifying methods (1/2)



(a) Distribution of features in training set

(b) Mapping from $\mathcal{Z}$ to features

(c) Mapping from $\mathcal{W}$ to features

Figure 6. Illustrative example with two factors of variation (image features, e.g., masculinity and hair length). (a) An example training set where some combination (e.g., long haired males) is missing. (b) This forces the mapping from $\mathcal{Z}$ to image features to become curved so that the forbidden combination disappears in $\mathcal{Z}$ to prevent the sampling of invalid combinations. (c) The learned mapping from $\mathcal{Z}$ to $\mathcal{W}$ is able to "undo" much of the warping.

Perceptual path length

$$l_{\mathcal{Z}} = \mathbb{E}\Big[\frac{1}{\epsilon^2} d\big(G(\text{slerp}(\mathbf{z}_1, \mathbf{z}_2; t)), G(\text{slerp}(\mathbf{z}_1, \mathbf{z}_2; t+\epsilon))\big)\Big], \quad (2)$$

$$l_{\mathcal{W}} = \mathbb{E}\Big[\frac{1}{\epsilon^2} d\big(g(\text{lerp}(f(\mathbf{z}_1), f(\mathbf{z}_2); t)), g(\text{lerp}(f(\mathbf{z}_1), f(\mathbf{z}_2); t+\epsilon))\big)\Big], \quad (3)$$

Linear separability

- Labeled genearted images
- Classify them using the auxiliary classification network
- Fit a linear SVM to predict the label based on the latent-space point and classify the points by this plan
- compute the conditional entropy

Disentanglement studies & Quantifying methods (2/2)

	Method	Path length full	Path length end	Separability
B	Traditional generator $\mathcal{Z}$	412.0	415.3	10.78
D	Style-based generator $\mathcal{W}$	446.2	376.6	3.61
E	+ Add noise inputs $\mathcal{W}$	**200.5**	**160.6**	3.54
	+ Mixing 50% $\mathcal{W}$	231.5	182.1	**3.51**
F	+ Mixing 90% $\mathcal{W}$	234.0	195.9	3.79

Table 3. Perceptual path lengths and separability scores for various generator architectures in FFHQ (lower is better). We perform the measurements in $\mathcal{Z}$ for the traditional network, and in $\mathcal{W}$ for style-based ones. Making the network resistant to style mixing appears to distort the intermediate latent space $\mathcal{W}$ somewhat. We hypothesize that mixing makes it more difficult for $\mathcal{W}$ to efficiently encode factors of variation that span multiple scales.

	Method	FID	Path length full	Path length end	Separability
B	Traditional 0 $\mathcal{Z}$	5.25	412.0	415.3	10.78
	Traditional 8 $\mathcal{Z}$	4.87	896.2	902.0	170.29
	Traditional 8 $\mathcal{W}$	4.87	324.5	212.2	6.52
	Style-based 0 $\mathcal{Z}$	5.06	283.5	285.5	9.88
	Style-based 1 $\mathcal{W}$	4.60	219.9	209.4	6.81
	Style-based 2 $\mathcal{W}$	4.43	**217.8**	199.9	6.25
F	Style-based 8 $\mathcal{W}$	**4.40**	234.0	**195.9**	**3.79**

Table 4. The effect of a mapping network in FFHQ. The number in method name indicates the depth of the mapping network. We see that FID, separability, and path length all benefit from having a mapping network, and this holds for both style-based and traditional generator architectures. Furthermore, a deeper mapping network generally performs better than a shallow one.

Conclusion

- Proposed new generator
  - Separation of high-level attributes and stochastic effects
  - The linearity of the intermediate latent space
- Proposed two new methods to quantify interpolation quality and disentanglement
  - Perceptual path length and linear separability
- Introduced a new, highly varied and high-quality dataset of human faces

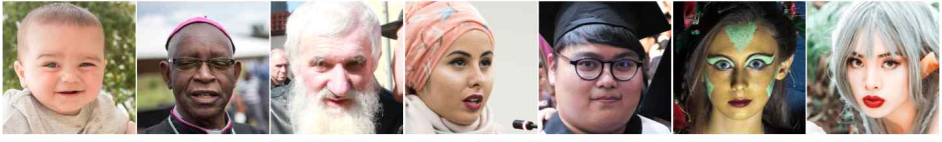


Figure 7. The FFHQ dataset offers a lot of variety in terms of age, ethnicity, viewpoint, lighting, and image background.

Thank you

References

[1] Karras, Tero, Samuli Laine, and Timo Aila, "A style-based generator architecture for generative adversarial networks," Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition, 2019.

[2] Ioffe, Sergey, and Christian Szegedy, "Batch normalization: Accelerating deep network training by reducing internal covariate shift," International conference on machine learning. PMLR, 2015.

[3] Ulyanov, Dmitry, Andrea Vedaldi, and Victor Lempitsky, "Instance normalization: The missing ingredient for fast stylization," arXiv preprint arXiv:1607.08022, 2016.

[4] Wu, Yuxin, and Kaiming He, "Group normalization," Proceedings of the European conference on computer vision (ECCV), 2018.

[5] Karras, Tero, et al., "Progressive growing of gans for improved quality, stability, and variation," arXiv preprint arXiv:1710.10196, 2017.

Supplementary Materials

A Trick for Visualization: Truncation Trick

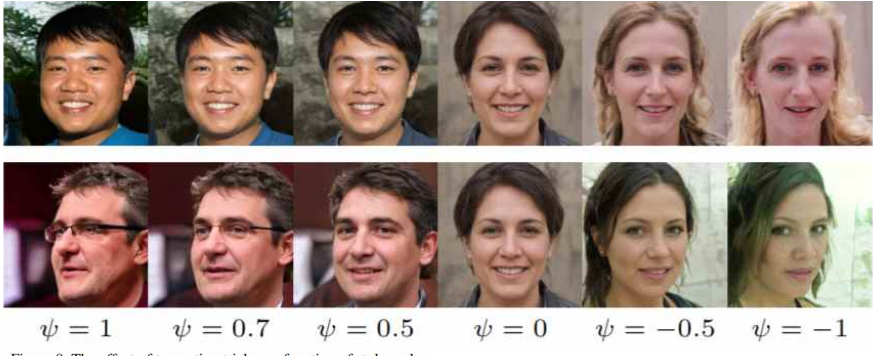


Figure 8. The effect of truncation trick as a function of style scale ψ. When we fade $\psi \to 0$, all faces converge to the "mean" face of FFHQ. This face is similar for all trained networks, and the interpolation towards it never seems to cause artifacts. By applying negative scaling to styles, we get the corresponding opposite or "anti-face". It is interesting that various high-level attributes often flip between the opposites, including viewpoint, glasses, age, coloring, hair length, and often gender.